

Dining Philosophers Café

Visual Deadlock, Starvation & Concurrency Workbench with Tri-Lingual Audio Narration

COURSE

Operating Systems (CS301)

AUTHOR / LEAD

Keyur Rana (Ranakeyur-31525)

REPOSITORY

-Dining-Philosophers-Cafe

STATUS & STACK

Fullstack (React 19 + Express)

Executive Project Abstract

The **Dining Philosophers Problem** is an iconic synchronization benchmark formulated by Edsger W. Dijkstra in 1965 to model resource contention, concurrency hazards, and deadlock vulnerability in multiprocessing operating systems. **Dining Philosophers Café** is a state-of-the-art interactive fullstack educational laboratory engineered with React 19, Tailwind CSS, Lucide icons, Node.js & Express SSE backend, and HTML5 Web Speech & Web Audio APIs.

This document delivers a complete theoretical and practical project dissection: exploring all 4 Coffman conditions, Havender's Resource Ordering hierarchy proof, Starvation conspiracy mechanisms with fair FIFO aging resolution, POSIX Mutexes & Dijkstra Invariant verification across 10,000 cycles, and comprehensive viva-voce preparation.

1. Classical Problem Formulation & Theoretical Foundations

Five philosophers sit around a circular table. In front of each is a bowl of noodles. Between every adjacent pair of philosophers lies exactly one shared fork (total 5 forks: F_0, F_1, F_2, F_3, F_4).

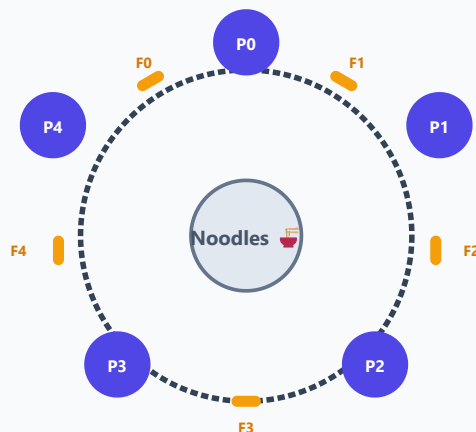


Figure 1: Circular Dining Philosophers Topology ($N=5$ Processors and Shared Mutex Resources)

1.1 State Transitions of a Philosopher

A philosopher P_i switches between three classic lifecycle states:

- **THINKING:** Independent CPU computing; requires no external resources.
- **HUNGRY:** Concurrently competing to acquire both left fork F_i and right fork $F_{(i+1)\%5}$.
- **EATING:** Entering the Critical Section; holding both forks exclusively, consuming noodles, and eventually releasing both forks back to the table.

1.2 The Four Coffman Conditions (1971)

Deadlock in any computing system can arise if and only if all four conditions hold simultaneously:

Condition	Formal Definition	Café Metaphor	Mitigation in this Project
1. Mutual Exclusion	Each resource unit can be assigned to at most one process at any instant.	Only one person can hold a physical fork; two diners cannot grab the same fork simultaneously.	Inherent to physical forks; protected by POSIX binary mutex <code>pthread_mutex_t</code> .
2. Hold and Wait	A process currently holding at least one resource is permitted to request other resources held by peers.	Holding your left fork tightly in your hand while stubbornly waiting for your neighbor to release their fork.	Tanenbaum Monitor: Acquire BOTH forks atomically (all-or-nothing), or release current holds.
3. No Preemption	Resources cannot be forcibly confiscated; they must be released voluntarily by the holding process.	You cannot physically slap the fork out of your neighbor's hand! They must put it down willingly.	Preemption Protocol: If the 2nd fork cannot be acquired within a timeout, drop the 1st fork!
4. Circular Wait	A closed sequence of processes $\{P_0, P_1, \dots, P_{n-1}\}$ exists where each P_i is waiting for a resource held by $P_{(i+1)\%n}$.	$P_0 \rightarrow F_1 \rightarrow P_1 \rightarrow F_2 \rightarrow P_2 \rightarrow F_3 \rightarrow P_3 \rightarrow F_4 \rightarrow P_4 \rightarrow F_0 \rightarrow P_0$. A closed ring of blocked threads!	Havender's Resource Ordering: Enforce strict global hierarchy $F_0 < F_1 < F_2 < F_3 < F_4$. The cycle cannot form!

2. Algorithmic Solutions & Formal Proofs

2.1 The Naive Greedy Protocol (Guaranteed Deadlock)

In the naive protocol, each philosopher follows the identical symmetrical program:

```
// Symmetrical Naive Algorithm (Vulnerable to Deadlock)
void philosopher(int i) {
    while (true) {
        think();
        pthread_mutex_lock(&fork[i]);           // Pick up Left Fork
        pthread_mutex_lock(&fork[(i + 1) % 5]); // Pick up Right Fork
        eat();
        pthread_mutex_unlock(&fork[(i + 1) % 5]); // Release Right Fork
        pthread_mutex_unlock(&fork[i]);          // Release Left Fork
    }
}
```

⚠ The Deadlock Execution Trace

Under simultaneous context switching (e.g., at time t_0), all 5 philosophers execute statement 1 and acquire their left fork. At time t_1 , all 5 attempt statement 2 to acquire their right fork. Since every fork is already locked, every philosopher blocks indefinitely. System throughput drops to **0%**, and the Wait-For Graph contains the cycle:

`P0 → P1 → P2 → P3 → P4 → P0` .

2.2 Havender's Total Resource Ordering (Hierarchy Protocol)

Proposed by J. W. Havender in 1968, this solution breaks **Circular Wait** by assigning a global strict total order to resources: $\$R(F_i) = i\$$. Each philosopher is required to request their assigned forks in *strictly ascending order*: $\$\text{First fork} = \min(F_{\text{left}}, F_{\text{right}}), \quad \text{Second fork} = \max(F_{\text{left}}, F_{\text{right}})\$$

Philosopher	Left Fork	Right Fork	First Acquired (\$\min\$)	Second Acquired (\$\max\$)	Effect
Priya (P0)	$\$F_0\$$	$\$F_1\$$	$\$F_0\$$	$\$F_1\$$	Standard order
Arjun (P1)	$\$F_1\$$	$\$F_2\$$	$\$F_1\$$	$\$F_2\$$	Standard order
Dev (P2)	$\$F_2\$$	$\$F_3\$$	$\$F_2\$$	$\$F_3\$$	Standard order
Meera (P3)	$\$F_3\$$	$\$F_4\$$	$\$F_3\$$	$\$F_4\$$	Standard order
Kabir (P4)	$\$F_4\$$	$\$F_0\$$	$\$F_0\$ \text{ (Inverted!)} \$$	$\$F_4\$$	P4 acquires lower fork F0 first!

Formal Mathematical Proof of Deadlock-Freedom

Theorem: Havender's Resource Ordering protocol guarantees that a cyclic wait cannot exist.

Proof by Contradiction:

Assume a deadlock exists under this protocol. Then there must exist a directed cycle in the Wait-For Graph: $P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_k \rightarrow P_0$ where $P_i \rightarrow P_{i+1}$ denotes that P_i is blocked waiting for a resource held by P_{i+1} .

Let $r(P_i, R)$ be the index of resource R requested by P_i while holding another resource. Under the protocol, a process holding resource a can only request resource b if $b > a$.

Therefore: $\text{Index}(R_0) < \text{Index}(R_1) < \text{Index}(R_2) < \dots < \text{Index}(R_k) < \text{Index}(R_0)$ This implies $\text{Index}(R_0) < \text{Index}(R_0)$, which is a logical and mathematical contradiction.

Hence, no circular wait can form. The system is provably deadlock-free. $\blacksquare$

2.3 The Waiter Arbitrator Protocol (Pigeonhole Principle)

A central arbitrator (the café waiter) introduces a Counting Semaphore initialized to $N - 1 = 4$. At most 4 philosophers are granted permission to sit at the table simultaneously:

Pigeonhole Principle Deduction

By the Pigeonhole Principle, if $k \leq 4$ philosophers compete for $N = 5$ shared forks, the average number of available forks per seated diner is: $\frac{5}{4} = 1.25 > 1$ Consequently, at least one seated philosopher is guaranteed to acquire both their left and right forks, eat, and release them. Deadlock is impossible.

3. Starvation (Livelock) & Fair Aging Resolution

Deadlock and Starvation are fundamentally distinct concurrency hazards:

Metric / Aspect	Deadlock	Starvation (Livelock)
System State	Complete permanent freeze; zero system-wide progress.	System makes continuous progress, but one or more threads never make progress.
Thread Participation	All threads in the cycle are blocked waiting.	Some aggressive threads repeatedly succeed; the victim thread is starved.
CPU Utilization	0% (all threads in blocked wait state).	Active (threads eating, thinking, and context-switching).
Detection Method	Cycle in Wait-For Graph (Tarjan's algorithm / DFS).	Wait-time threshold timers ($\Delta t > t_{\text{threshold}}$).
Resolution	Deadlock prevention, preemption, process termination.	Fair FIFO queues, Aging priority schemes, lottery scheduling.

3.1 The Neighbor Conspiracy Scenario

Consider philosopher **Arjun (P1)** sitting between **Priya (P0)** and **Dev (P2)**. P0 and P2 alternate eating in an aggressive, out-of-phase cycle:

- At t_0 , P0 eats using forks F_0 and F_1 . P1 is hungry but blocked because F_1 is held by P0.
- At t_1 , P0 finishes eating and releases F_1 . But simultaneously, P2 begins eating using F_2 and F_3 !
- Now, P1 attempts to eat, but fork F_2 is held by P2!
- At t_2 , P2 finishes eating and releases F_2 , but P0 immediately grabs F_1 again!

In this infinite ping-pong conspiracy, forks F_1 and F_2 are never simultaneously free. P1 starves indefinitely despite the system operating at full throughput!

3.2 Fair FIFO Aging Queue Solution

How the FIFO Aging Mechanism Solves Starvation

In Fair Mode, whenever a philosopher becomes hungry, they take a timestamped queue ticket. When neighbor P0 releases fork F_1 , the system checks the queue: **P1 has a higher wait-time priority than P0!** P0 is prohibited from re-entering the table until P1's request is serviced. Wait-time aging dynamically escalates P1's priority, guaranteeing bounded wait time: $\text{WaitTime}(P_i) \leq 2 \times T_{\text{eat}}$

4. System Architecture & Technology Stack

Frontend Application

React 19 + Vite: Componentized UI architecture using modern hooks (`useDeadlockSim`, `useSimulation`).

Tailwind CSS: Custom theme tokens, dark mode classes, glowing indicators, animated SVGs.

Lucide React: Crisp vector iconography for all concurrency states and controls.

Backend & SSE Server

Node.js & Express: High-performance server on port 3000.

Server-Sent Events (SSE): Real-time unidirectional streaming from `/api/events`.

REST APIs: Endpoints for manual simulation step control and viva educational content.

Tri-Lingual Audio Engine

Web Speech API: Native `window.speechSynthesis` supporting English (`en-US`), Hindi (`hi-IN`), and Gujarati (`gu-IN`).

Real-time audio ticker displaying spoken subtitles and audio state.

Web Audio API Synthesizer

Hardware Synthesis: Pure sinusoidal oscillator synthesis without external audio files.

Distinct sound signatures for fork pickup (800Hz), eating chomps (240Hz), deadlock buzzer (120Hz saw), and victory fanfare.

5. Walkthrough of the 6 Laboratory Modules

5.1 Module 1: Dashboard (`/`)

The command center of the laboratory. Provides a comprehensive hero section, problem overview, interactive dining table preview, and high-level architecture diagram. Introduces users to the café theme with avatars: Priya (Student), Arjun (Developer), Dev (Artist), Meera (Scientist), and Kabir (Professor).

5.2 Module 2: Deadlock Lab (`/deadlock`)

An interactive 10-step step-by-step simulator detailing exactly how deadlock develops tick by tick:

- **Live Directed Wait-For Graph (WFG):** Visualizes processes as nodes and held/requested resources as directed edges, highlighting the red cycle when deadlock occurs.
- **Live Coffman Condition Matrix:** Real-time badges for Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait.
- **Step Scrubbing:** Play, pause, step forward, and reset controls allowing students to analyze each micro-transition.

5.3 Module 3: Solution Lab (`/solution`)

Side-by-side comparative simulation comparing the **Naive Greedy Protocol** against **Havender's Resource Ordering Protocol**. Demonstrates in real time how Kabir (P4) picking fork F_0 first prevents the circular wait from closing, freeing F_4 for Meera (P3) to eat safely.

5.4 Module 4: Starvation Lab (`/starvation`)

Simulates the subtle distinction between Deadlock (system frozen) and Starvation (one thread frozen while others eat). Features a live **Hunger Meter (0–100%)** for Arjun (P1), an interactive conspiracy generator, and a toggleable **Fair FIFO Queue** that enforces fairness and resets hunger levels to 0%.

5.5 Module 5: Synchronization Lab (`/synchronization`)

Deep dive into OS-level concurrency primitives:

- **POSIX Mutex Visualizer:** Displays `pthread_mutex_lock()` and `unlock()` operations on each fork.
- **Counting Semaphore Panel:** Models the Waiter Arbitrator with permit counting ($k=4$).
- **Dijkstra Invariant Automated Stress Test:** Runs an automated 10,000-iteration Monte Carlo stress test checking that no two adjacent philosophers eat simultaneously ($\sum \text{eating}(P_i, P_{i+1}) = 0$).

5.6 Module 6: Telemetry & Lab Report (`/results`)

Aggregates full-session telemetry including total meals eaten, throughput (meals/sec), deadlock incident counts, starvation occurrences, and a printable academic lab evaluation sheet ready for submission.

6. Operating System Viva Voce & Interview Guide

Q1: What is the fundamental difference between Deadlock and Starvation?

In **Deadlock**, all processes in a cyclic wait are blocked; no CPU progress is made system-wide. In **Starvation**, system progress is actively occurring (other processes eat repeatedly), but one specific process is perpetually deprived of required resources due to scheduling bias or unfairness.

Q2: Why does Havender's Resource Ordering guarantee freedom from deadlock?

By enforcing a strict total order ($F_0 < F_1 < F_2 < F_3 < F_4$), a circular dependency chain cannot mathematically exist. A cycle in a Wait-For Graph requires at least one transition from a higher-indexed resource to a lower-indexed resource while holding the higher one, which Havender's rule strictly prohibits.

Q3: How does the Waiter solution leverage the Pigeonhole Principle?

With 5 forks and at most 4 seated philosophers (enforced by a counting semaphore initialized to 4), the pigeonhole distribution $\lceil 5 / 4 \rceil = 2$ guarantees that at least one philosopher will receive both forks, allowing them to eat and release resources back to the table.

Q4: What is Dijkstra's Safety Invariant in the Dining Philosophers Problem?

For all $i \in \{0, \dots, N-1\}$: if $\text{State}(P_i) == \text{EATING}$, then $\text{State}(P_{(i-1)\%N}) \neq \text{EATING}$ and $\text{State}(P_{(i+1)\%N}) \neq \text{EATING}$. Two adjacent philosophers can never be in the EATING state simultaneously because each fork is mutually exclusive.

Q5: What is the difference between a Binary Semaphore and a Mutex?

A **Mutex** enforces ownership: only the thread that locked the mutex is legally permitted to unlock it. A **Binary Semaphore** has no concept of ownership: any thread can execute `sem_post()` to wake up a thread blocked on `sem_wait()`, making it suitable for signaling between threads.

Q6: What is a Futex and why is it preferred in modern Linux kernels?

A **Futex** (Fast Userspace Mutex) optimizes uncontended locks completely in user space using atomic CPU instructions (like `compare-and-swap`), avoiding expensive kernel system calls unless lock contention actually occurs.

7. Setup, Build, and Execution Manual

7.1 Prerequisites

- Node.js (v18.0.0 or higher recommended)
- npm (v9.0.0 or higher)
- Modern web browser (Google Chrome, Microsoft Edge, Mozilla Firefox) with Web Speech API support

7.2 Running in Development Mode

```
# 1. Install dependencies
npm install

# 2. Start Vite Hot Module Replacement (HMR) dev server
npm run dev

# Vite runs at: http://localhost:5173
```

7.3 Running the Fullstack Express Server

```
# 1. Compile production bundle
npm run build

# 2. Launch Express server with REST API + SSE
npm start

# Server listens on: http://localhost:3000
```

7.4 REST & SSE API Reference

Endpoint	Method	Description
/api/status	GET	Returns JSON server telemetry, connected client count, and philosopher states.
/api/events	GET	Server-Sent Events (SSE) stream broadcasting live simulation state changes.
/api/action	POST	Dispatches manual philosopher actions (take_left , take_right , eat , think).
/api/scenarios/reset	POST	Resets simulation state, releases all forks, and clears deadlock flags.
/api/education	GET	Delivers Coffman condition definitions and viva questions for educational client consumption.